

Neural Networks

Lecture 1 — The Single Neuron

“You’ve fit lines to data with gradient descent and clustered galaxies without labels. Now we build the machines that learn representations.”

Where we are

- **SL week:** linear regression, SVMs, random forests, cross-validation
- **UL week:** covariance, distance metrics, loss functions, gradient descent, PCA, clustering
- **This week:** neural networks — gradient descent applied to compositions of functions

Note

Everything this week builds on gradient descent from the UL notebooks. If that feels shaky, the UL notebook 03 is your reference.

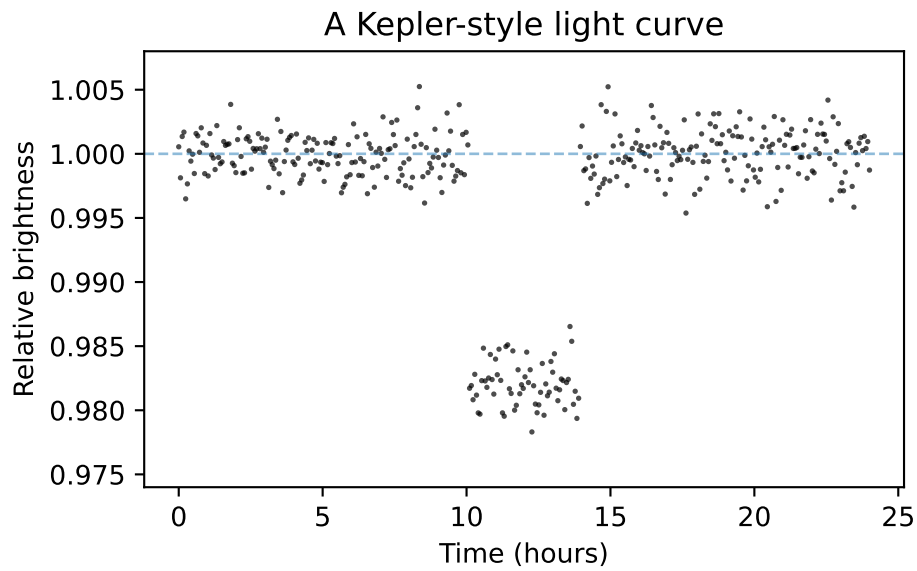
A detection problem

NASA’s Kepler telescope stared at 150,000 stars for four years, recording brightness every 30 minutes.

A real exoplanet transit produces:

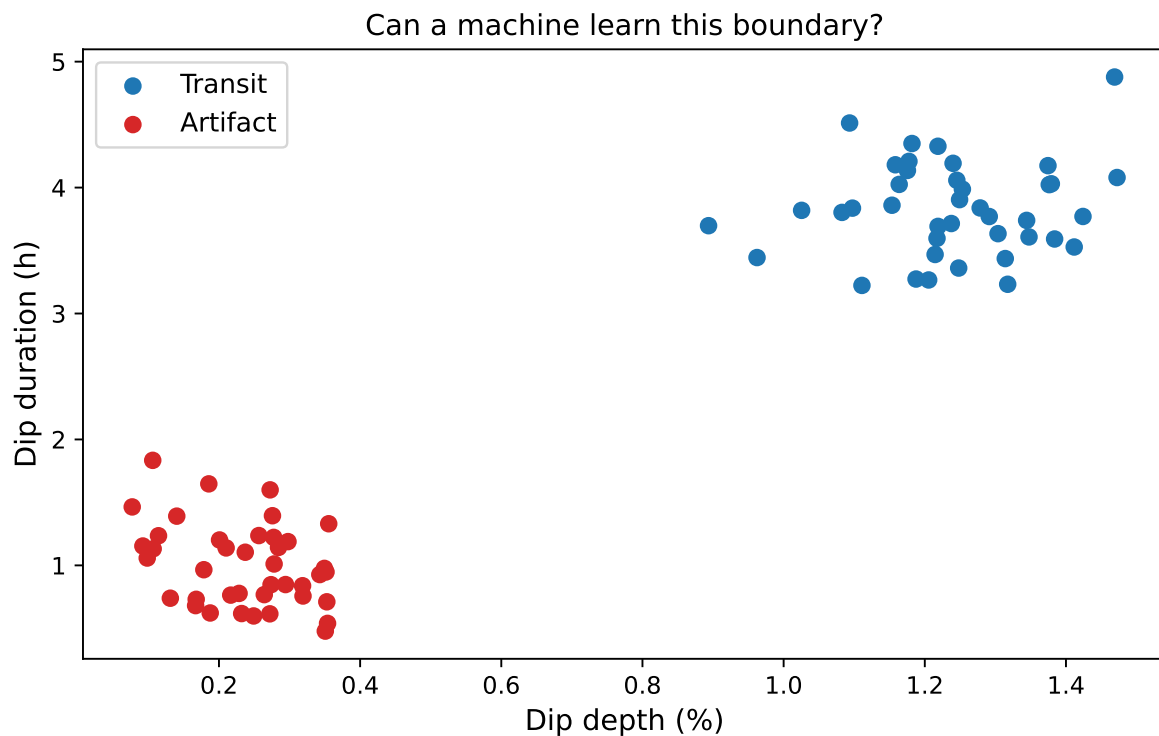
- A brief, periodic dip in flux
- Depth set by $(R_{\text{planet}}/R_{\text{star}})^2$
- Duration set by orbital speed and geometry

The problem: most dips are noise, systematics, or eclipsing binaries misidentified as planets.
Can we build a detector that learns to tell them apart?



Two features, two classes

A single transit event can be summarised by two numbers: how deep and how long.



The classes look separable — there should be a line that divides them. A single neuron draws exactly one line.

The neuron

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

Symbol	Name	Role
$\mathbf{x}$	input	dip depth, duration
$\mathbf{w}$	weights	how much each feature matters
b	bias	shifts the threshold
σ	activation	squashes output to (0,1)
y	output	probability: transit or artifact?

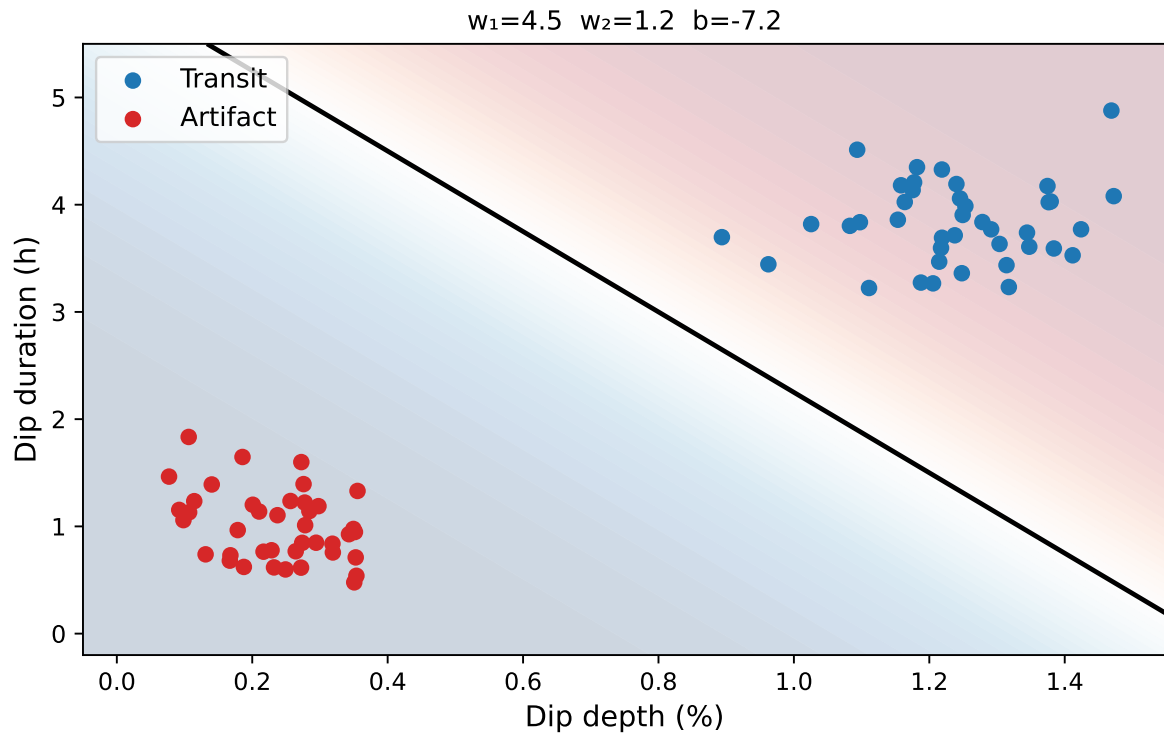
```
import torch

x = torch.tensor([1.2, 4.0]) # one transit
w = torch.tensor([3.0, 1.0])
b = torch.tensor(-5.0)
```

```
z = torch.dot(w, x) + b      # weighted sum
y = torch.sigmoid(z)         # → probability
```

The decision boundary is where $\mathbf{w} \cdot \mathbf{x} + b = 0$ — where the neuron outputs exactly 0.5.

Finding the boundary



In the notebook you will drag w_1 , w_2 , b to find the best boundary yourself.

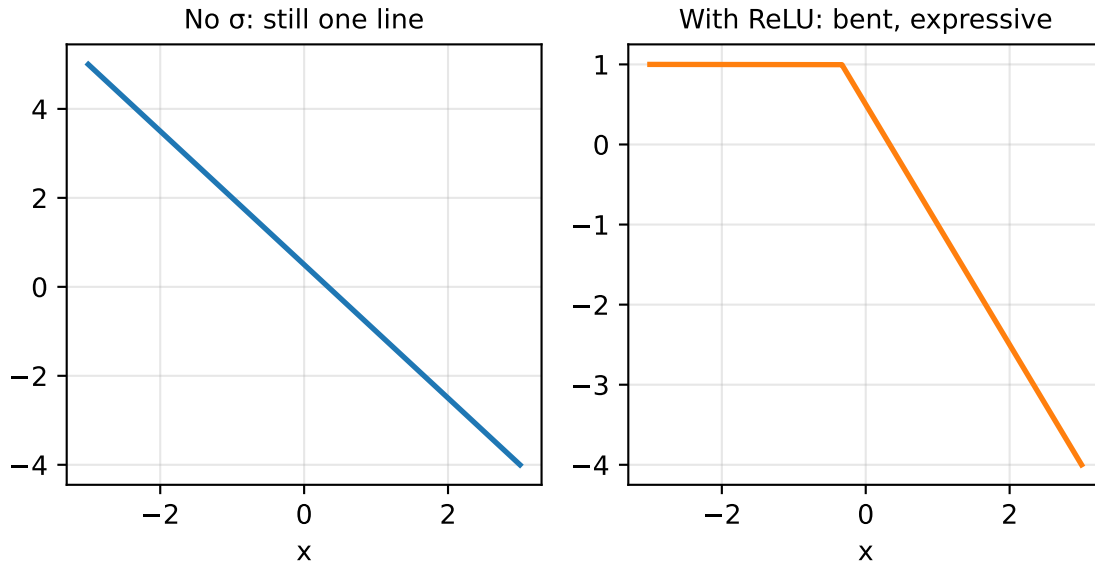
Why do we need σ ?

Without an activation function, stacking layers gives nothing new:

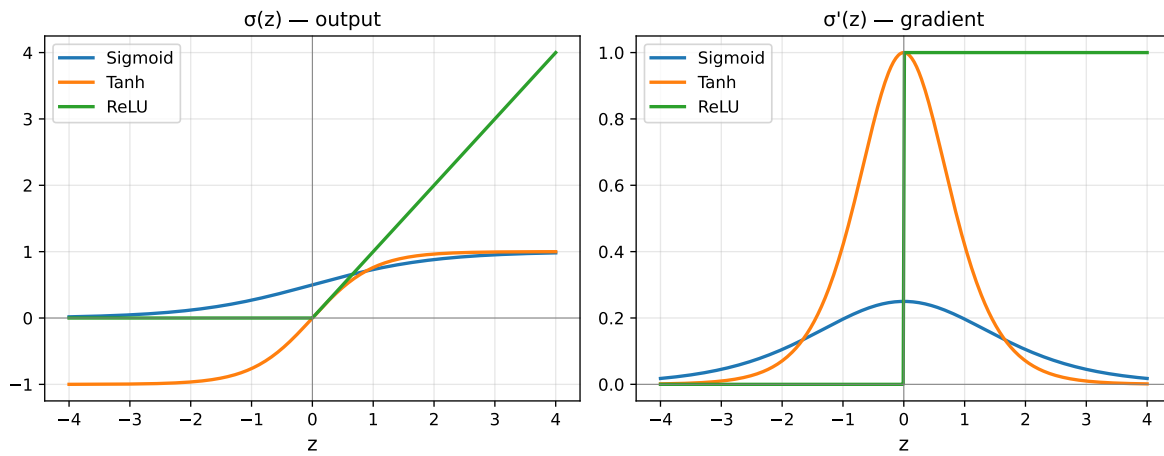
$$W_2(W_1\mathbf{x} + b_1) + b_2 = \underbrace{(W_2W_1)}_{\text{one matrix}}\mathbf{x} + \underbrace{(W_2b_1 + b_2)}_{\text{one bias}}$$

Depth buys you nothing if every layer is linear — the whole network collapses to a single linear transformation.

σ breaks this: compositions of layers become genuinely more expressive.



Activation functions



The gradient column is what flows backwards during training. Where $\sigma'(z) \approx 0$, learning stops.

The vanishing gradient problem

Sigmoid and tanh both saturate.

When $|z|$ is large, $\sigma'(z) \approx 0$. Gradients multiplied through many saturated layers shrink exponentially — the network stops learning.

ReLU fixes this for positive inputs: $\sigma'(z) = 1$ exactly, so gradients pass through unchanged.

The cost: a ReLU neuron whose input is always negative outputs 0, has gradient 0, and never updates. This is a dead neuron.

Function	Gradient range	Saturates?
Sigmoid	$(0, 0.25)$	Yes
Tanh	$(0, 1)$	Yes
ReLU	$\{0, 1\}$	Half-domain
Leaky ReLU	$\{0.01, 1\}$	No

In practice: **ReLU** for hidden layers, **sigmoid** for binary output.

In the notebook, Part 2 lets you compare all five functions and their gradients interactively.

The neuron draws one line.

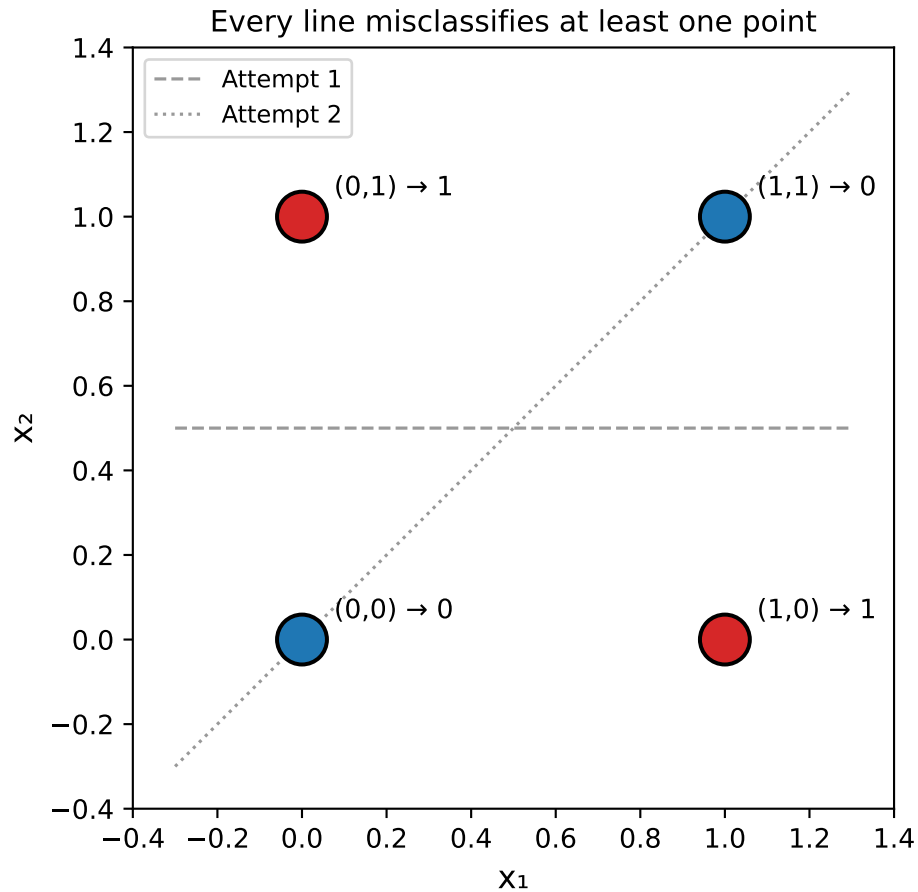
What if one line is not enough?

XOR — the hard limit

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

The two class-0 points sit at **opposite corners**. Any line that separates them cuts through the class-1 points.

No linear boundary can solve this — for any weights.



What would it take?

The XOR points are not linearly separable — but they **are** separable if you could draw two lines.

Two lines divide the plane into regions. With the right arrangement, all four XOR points land in the correct region.

One neuron = one line.

Two neurons in a hidden layer = two lines.

This is the motivation for multilayer networks — the subject of the next lecture.

Tip

Open the notebook. Part 4 lets you try every possible weight combination on XOR. Can you get 4/4? At the end, try the Part B challenge: design a $2 \rightarrow 2 \rightarrow 1$ network by hand that solves it.

References

Papers

- McCulloch & Pitts (1943) — *A logical calculus of the ideas immanent in nervous activity*. Bulletin of Mathematical Biophysics 5, 115–133
- Rosenblatt (1958) — *The Perceptron: a probabilistic model for information storage and organization in the brain*. Psychological Review 65(6), 386–408

Videos & interactive

- 3Blue1Brown — *But what is a neural network?* [youtube.com/watch?v=aircArvnKk](https://www.youtube.com/watch?v=aircArvnKk)
- Karpathy — *Hacker's guide to Neural Networks* karpathy.github.io/neuralnets/
- TensorFlow Playground — playground.tensorflow.org